Development Plan for AquaFlow Water Meter Reading and Billing System

How to use this plan

Use the milestones in order. Finish the required work and tests for one milestone before moving to the next major part. Keep screenshots, test results, diagrams, decisions, and other proof of completed work in the evidence folders. If a requirement is unclear, document the assumption or decision instead of silently choosing a behavior.

Practical development approach

The project should be developed in small working parts. First make the basic backend services work, then connect them, then build the user interfaces, and only after that move the complete application to Kubernetes. This reduces the chance of having many unfinished parts at the same time.

Recommended working routine

- 1. Read the current milestone and identify its required output before writing code.
- 2. Learn the concepts needed for that task and write short notes in docs/journal.md.
- 3. Complete a small practice exercise that is separate from the main application when the topic is new.
- 4. Implement the feature in AquaFlow using the documented service boundaries and business rules.
- 5. Run the relevant tests, review security and validation, update documentation, and capture evidence.
- 6. Mark the task complete only when its Definition of Done is satisfied; otherwise record the blocker and next action.

Progress status legend

Status	Meaning	When to use it
Not started	No work has begun.	The task is still waiting in the plan.
Learning	Reading or practicing the required concept.	You can explain the concept but have not applied it fully.
In progress	Implementation or testing is underway.	Some work exists, but completion evidence is missing.
Blocked	Cannot continue because a decision, dependency, or error is unresolved.	Record the blocker, owner/action, and next review date.
Done	All task-level completion criteria pass.	Tests, documentation, security checks, and evidence are available.

Recommended rule: do not mark a task “Done” just because code has been written. A task is done when the behavior works, expected failure cases are checked, and another person can follow the documentation or reproduce the test.

29-Week Development Schedule

This schedule maps the 13 milestones in the detailed plan into an estimated 29-week sequence. It is a planning baseline, not a promise that every milestone will take exactly the stated time. If a milestone is delayed, adjust later weeks rather than skipping required tests or business-rule checks.

Week	Milestone	Main outputs	Exit check
Weeks 1–2	M0 — Orientation and domain discovery	Glossary, user stories, context map, billing spreadsheet, assumptions, ADR-0001, repository and journal.	Checkpoint A passed; core rules and open questions are documented.
Weeks 3–4	M1 — Monorepo and Customer Service	Workspace/tooling, PostgreSQL Compose, customer/account/meter models and endpoints, validation, tests and OpenAPI.	Fresh clone starts the service and tests pass.
Weeks 5–6	M2 — Auth and RBAC	Auth database, login/session flows, token verification, permissions, audit events and role tests.	Permission-matrix tests pass for all roles and protected endpoints.
Weeks 7–8	M3 — Gateway and containerization	Dockerfiles, Compose, gateway routing, health checks, request IDs, rate limits and non-root containers.	Fresh clone starts healthy; dependency failure is handled safely.
Weeks 9–10	M4 — Meter Reading Service	Reading rules R1–R7, submission, photos, estimated reads, rollover/replacement rules and review flow.	Reading scenarios and authorization tests pass.
Weeks 11–13	M5 — Billing Engine	Tariffs, pure calculator, rounding, bill runs, arrears, penalties, immutable bills, PDFs and ledger.	Three mandatory totals match exactly; key billing edge cases pass.
Weeks 14–15	M6 — Staff portal	Secure staff interface, customer/meter screens, reading review, tariffs, bill-run wizard and browser tests.	Staff can complete core workflows with role-based access.
Weeks 16–18	M7 — Events, payments and notifications	RabbitMQ, contracts, outbox/inbox, idempotency, payment flows, ledger credit and notification retries/DLQ.	Duplicate events do not duplicate payment or ledger effects.
Weeks 19–20	M8 — Customer portal and reader PWA	Customer bills and payments, reader offline queue, sync/conflict handling and mobile tests.	Offline route scenario and account-isolation tests pass.
Weeks 21–22	M9 — Quality gates	CI workflows, contract/E2E tests, image/dependency/secret scanning and protected branches.	Pull requests are blocked when required quality checks fail.
Weeks 23–25	M10 — Kubernetes and IaC	kind, Helm, secrets/config, probes, resource limits, Argo CD and rollback practice.	The application can be deployed and recovered in local Kubernetes.
Weeks 26–27	M11 — Observability and resilience	OpenTelemetry, metrics, logs, traces, dashboards, SLOs, alerts and failure exercises.	A cross-service failure can be detected and diagnosed using telemetry.
Weeks 28–29	M12 — Hardening, performance and capstone	Security review, load tests, backup/restore, full capstone run, known limitations and presentation.	Capstone passes repeatably; evidence and runbooks are ready.

Weekly review: at the end of each week, record what was completed, what failed, what you learned, which evidence was saved, and the first task for the next week. Keep optional work behind required milestones.

Milestone Dependency Map

Use this map to decide what to work on next. A milestone should not be treated as ready merely because its calendar week has arrived; first check that its prerequisites are complete.

Milestone	Prerequisite	Ready-to-start check
M0 — Domain discovery	Start here	Business rules, glossary, user stories, assumptions and service boundaries are understood.
M1 — Customer Service	M0	Domain objects and service ownership are documented.
M2 — Auth and RBAC	M1 foundation	Customer/account model exists so permissions can be applied to real resources.
M3 — Gateway and containers	M1 + M2 foundations	Services can run locally and have stable health/configuration behavior.
M4 — Reading Service	M1 + M2 + M3 foundations	Customer/meter data, role checks and local infrastructure are available.
M5 — Billing Engine	M4 reading rules and consumption contract	Reading outputs and tariff rules are stable; billing totals are independently verified.
M6 — Staff portal	Stable APIs from M1–M5	Core staff workflows can be driven by tested backend contracts.
M7 — Events, payments, notifications	M5 billing model + stable event contracts	Bill issuance and ledger rules are clear before asynchronous processing is added.
M8 — Customer portal and reader PWA	M4 + M5 + M7 APIs/workflows	Reading sync, bill display and payment status have working backend behavior.
M9 — Quality gates	Working services and repeatable local stack	CI can test actual application behavior and build known-good images.
M10 — Kubernetes and IaC	M9 build pipeline + Compose stack	Images and configuration are reproducible before deployment complexity is added.
M11 — Observability and resilience	Services deployed with stable request/event flow	Telemetry can follow real requests and messages across service boundaries.
M12 — Hardening and capstone	M0–M11 required outputs	End-to-end workflow is stable enough for security, load, recovery and final demonstration.

Dependency flow at a glance

Domain rules (M0) → Customer data (M1) → Authentication and permissions (M2) → Gateway and local containers (M3) → Meter readings (M4) → Billing (M5) → Staff workflow (M6) → Events, payments and notifications (M7) → Customer portal and offline reader (M8) → CI quality gates (M9) → Kubernetes deployment (M10) → Observability and resilience (M11) → Security, performance, recovery and capstone (M12).

Some work can overlap after its contracts are stable, but do not bypass a dependency that supplies required data, permissions, or business rules. If you must change an earlier contract, update the dependent tasks and tests as well.

Technology Learning, Practice and Implementation Checklist

For each technology, complete all four stages. “Learn” means you can explain the main idea in your own words. “Practice” means you have completed a small exercise. “Implement” means you have used it in AquaFlow. “Prove” means you have test output, a demonstration, or documentation showing that it works. Mark the checkboxes in your working copy as you progress.

Technology	Learn	Practice	Implement in AquaFlow	Proof of understanding
Git/GitHub	Repository, commits, branches, pull requests and conflict resolution.	Create a branch, commit a change, open a PR and resolve a small conflict.	Use branch/commit policy and PR review for AquaFlow changes.	Clean history, reviewed PR and repository instructions.
Node.js + TypeScript	Async code, modules, types, errors, environment variables and strict type checking.	Build a small typed API handler and validate input.	Use strict TypeScript and startup configuration validation in services.	Typecheck and tests pass; explain the request flow.
HTTP/API + Fastify	Methods, status codes, routing, middleware/hooks, pagination and error contracts.	Create a small CRUD API with validation and consistent errors.	Implement documented /api/v1 endpoints.	OpenAPI docs and API tests cover success and failure cases.
Zod validation	Schemas, parsing, refinements and safe error messages.	Validate a request body and reject invalid fields.	Validate API input, environment configuration and event payloads.	Tests prove valid input passes and invalid input is rejected.
PostgreSQL + Prisma	Tables, relations, constraints, migrations, indexes and transactions.	Model related tables and run a migration/rollback practice.	Use service-owned databases and migration workflow.	Migrations apply cleanly; constraints and integration tests pass.
Authentication + RBAC	JWT/JWKS, refresh-token rotation, permissions, session security and audit logs.	Create a role/permission matrix and test protected routes.	Apply backend authorization to every sensitive endpoint.	Positive and negative role tests pass; secrets are not logged.
Next.js + React	App Router, server/client components, forms, loading/error states and secure sessions.	Build a small form and list with loading and error states.	Implement staff/customer portals and protected workflows.	Playwright tests pass for key workflows and roles.
Docker + Compose	Images, layers, networks, volumes, health checks and environment configuration.	Containerize a small service and run it with a database.	Run the local AquaFlow stack with documented commands.	Fresh clone can start, inspect logs and stop the stack.
Redis	Cache/temporary data use, expiration and failure behavior.	Run a small expiring key example and test unavailable Redis.	Use only for the specific planned rate-limit/cache needs.	Behavior is documented and the system fails safely if relevant.
MinIO/object storage	Buckets, object metadata, pre-signed URLs and local S3-compatible storage.	Upload and retrieve a test file using a pre-signed URL.	Store reading photos and generated bill PDFs.	Access permissions and file lifecycle tests pass.
RabbitMQ/events	Queues, exchanges, acknowledgements, retries, dead-letter queues and event contracts.	Publish and consume a sample event, then test a failed consumer.	Connect services through typed events where specified.	Duplicate, malformed and failed messages behave as designed.

Technology	Learn	Practice	Implement in AquaFlow	Proof of understanding
Outbox/inbox/idempotency	Atomic event recording, duplicate detection and safe retries.	Simulate a repeated event and a crash between database commit and publish.	Prevent lost events and duplicate payment/ledger side effects.	Automated replay tests prove effects occur only once.
Vitest/Supertest/Testcontainers	Unit vs integration tests, fixtures, isolated dependencies and assertions.	Write unit tests and one database-backed integration test.	Test service rules and API behavior.	Tests are repeatable and run in the normal quality command.
Playwright/Pact/k6	Browser E2E, contract compatibility and load testing.	Write one browser test, one contract test and a small load script.	Test user journeys, service contracts and required load targets.	Results are saved and failures are investigated.
GitHub Actions + GHCR	CI jobs, secrets, artifacts, image tags and branch protection.	Create a workflow that runs lint and tests on a pull request.	Build and publish traceable images from approved changes.	Failed checks block merge; images map to a Git SHA.
Kubernetes + Helm + Argo CD	Pods, deployments, services, ingress, probes, resources, releases and rollback.	Deploy a small app to kind and perform a rollback.	Deploy the local AquaFlow stack through Helm/GitOps.	Repeatable deployment, health checks and rollback are demonstrated.
OpenTelemetry + Prometheus/Grafana/Loki/Tempo	Logs, metrics, traces, correlation IDs, SLIs/SLOs and alerts.	Trace one request and build a basic latency/error dashboard.	Instrument service and RabbitMQ paths.	A cross-service trace and useful failure alert are demonstrated.
Security + backups + recovery	IDOR, input limits, secret handling, dependency scanning, backup and restore.	Test access denial and restore a sample database backup.	Run security checks and restore the project database into a fresh environment.	Security test evidence and successful restore verification are saved.

Do not try to master every advanced feature before moving forward. Learn enough to complete the current task safely, then deepen your understanding as later milestones require more advanced use.

Task-Level Definition of Done and Evidence

The detailed milestone lists describe what to build. Use the following checklist for every task or small group of closely related tasks. A task is complete only when every relevant item is checked or explicitly marked “Not applicable” with a reason.

Check	Completion requirement	Evidence to save
☐ Goal clear	Task has an ID, purpose, expected behavior and acceptance criteria.	Task card, issue or project-board entry.
☐ Dependencies ready	Required API contracts, data, decisions and prior milestones are ready.	Linked dependency tasks or approved decision record.
☐ Learning/practice done	For unfamiliar topics, notes and a small practice exercise exist.	Short learning note, example or exercise result.
☐ Implementation works	The required behavior is implemented in the correct owning service.	Code change and a reproducible demo.
☐ Validation/errors covered	Invalid inputs, missing data, boundary conditions and expected failures are handled.	Unit/integration tests for positive and negative cases.
☐ Security/permissions checked	Authorization, data isolation, secret handling and sensitive logging are checked where relevant.	Permission tests or documented security review.

Check	Completion requirement	Evidence to save
☐ Automated checks pass	Relevant tests, lint, typecheck and integration/E2E checks pass.	Test output or CI run link.
☐ Observability added	Useful logs, request IDs, metrics or traces are included where relevant.	Example log/trace/metric or N/A reason.
☐ Documentation updated	API contracts, setup steps, diagrams, decisions and known limitations reflect the change.	Updated docs or ADR.
☐ Review and demo complete	Change is reviewed; behavior can be demonstrated or reproduced.	Approved PR, screenshot, command output or demo notes.
☐ Final status recorded	Task status, completion date and remaining follow-up are recorded.	Updated project board and journal.

Reusable task card

Field	Fill this in before or during implementation
Task ID / milestone	
Goal / expected behavior	
Prerequisites / dependencies	
Learning needed / practice exercise	
Implementation steps	
API, database or event changes	
Validation and security cases	
Tests and acceptance criteria	
Documentation and evidence location	
Status / blocker / next action	

Keep this task card in your issue tracker or project journal. For a very small task, combine related work into one card; for risky billing, payment, authentication, or data-migration work, use separate cards and stricter review.

Executive Development Plan

AquaFlow is a water meter reading and billing platform. The completed system should allow staff to manage customers, accounts, and meters; field readers to record readings even when offline; the system to check and approve readings; the billing service to calculate clear and traceable bills; the payment service to update the ledger only once; the notification service to inform customers; and customers to view and pay their bills.

The roadmap describes a target architecture of independent Node.js services, a Next.js web application, PostgreSQL databases, RabbitMQ events, Redis, MinIO, Mailpit, Kubernetes, CI/CD, and observability.

System scope

Area	Must be implemented
Identity & access	Staff/customer login; JWT access and refresh tokens; four roles; permission enforcement; audit of sensitive actions.
Customer management	Customers, service accounts, service addresses, classifications, statuses, zones/routes.
Meter management	Meter inventory, installation, replacement, installation history, current meter.
Field reading	Route assignment; billing periods; single/batch readings; photos; offline queue; validation; anomaly review; estimated reads.
Billing	Versioned tariffs; bill runs; tier calculations; fees/taxes; arrears; penalties; adjustments; immutable bills; PDF; ledger.
Payments	Over-the-counter and online/mock payments; webhooks; idempotency; receipts; pending/confirmed state.
Notifications	Bill issued, payment received, overdue, flagged-reading notifications; delivery log; local Mailpit.
Staff portal	Dashboard; customers/accounts; meter history; review queue; tariffs; bill-run wizard; users/roles; audit log.
Customer portal	Bills; bill details/PDF; usage history; online payment; payment status; notification preferences.
Reader PWA	Assigned route; offline reading capture; photos; queued submissions; sync and conflict handling.
Platform	Docker Compose; Kubernetes; GitOps; secrets; health checks; resource limits.
Operations	Logs, metrics, traces, SLOs, alerts, resilience experiments, backups and restore drill.

Required architecture rules

- Each backend service owns its own database. No service may query another service's tables.
- Cross-service integration uses synchronous APIs or asynchronous domain events.
- Business/domain logic stays inside the owning service; the gateway is not a business-logic layer.
- Money must not use JavaScript floating-point arithmetic.
- Issued bills are immutable. Corrections use adjustments/credit/debit memos.
- Every externally triggered mutation that can be retried must have an idempotency strategy where applicable.
- Backend authorization is main source of truth; hiding a UI button is never considered security.
- Secrets are never committed to Git.
- Services run as non-root containers and expose health/readiness endpoints.
- Every milestone has tests, documentation, a demo, and a Definition-of-Done review.

System Requirements and Business Rules

This section keeps the business rules from the original roadmap. These rules should be treated as the basis for the implementation unless an approved decision record changes them.

Core domain objects

Object	Definition / required behavior
Customer	Person or organization responsible for payment; may have multiple accounts.
Service account	Billable connection at one service address; has classification and status.
Classification	Residential, Commercial, Industrial, or Government; determines tariff.
Meter	Physical device with serial number, size and monotonically increasing register; may be replaced.
Zone / route	Geographic grouping of accounts; readers are assigned routes.
Billing period	Period with reading window, bill date and due date.
Reading	Meter register value on a date; consumption normally equals current minus previous.
Estimated reading	Generated when meter cannot be read; normally based on recent average.
Tariff	Versioned price schedule containing minimum charge, tiered rates, fees and taxes.
Bill	Immutable statement for one account and period.
Ledger	Running debit/credit history; balance is the sum of entries.
Arrears	Unpaid previous balance carried forward.
Penalty	Surcharge for overdue amounts after the defined grace period.
Rollover	Meter register passes its maximum and returns to zero.

2 Roles

Capability	Admin	Billing officer	Meter reader	Customer
Manage staff users & roles	Yes	-	-	-
Create/edit customers, accounts, meters	Yes	Yes	-	-
View accounts & meters	Yes	Yes	Own route	Own only
Submit meter readings	Yes	-	Own route	Stretch/self-read
Review/approve flagged readings	Yes	Yes	-	-

Capability	Admin	Billing officer	Meter reader	Customer
Manage tariffs	Yes	-	-	-
Run billing / issue bills / adjustments	Yes	Yes	-	-
View bills & ledger	Yes	Yes	-	Own only
Record OTC payments	Yes	Yes	-	-
Pay online	-	-	-	Own only
Audit log & dashboards	Yes	Dashboards	-	-

Reading validation rules

ID	Rule	System action
R1	Current reading lower than previous	Flag NEGATIVE unless rollover or meter replacement explains it.
R2	Consumption > 300% of average of last 3 periods	Flag HIGH_USAGE.
R3	Zero consumption on active account for 2 consecutive periods	Flag ZERO_USAGE.
R4	Reading date outside billing period reading window	Reject with clear error.
R5	Duplicate meter + period submission	Idempotently return existing reading; create nothing.
R6	No-access code	Create ESTIMATED reading from 3-period average.
R7	Flagged reading has no photo	Warn reader in app before submission.

Billing tariff supplied by the roadmap

Component	Rule
Minimum charge	First 10 m^3; charged even when consumption is lower: $250 flat.
Block 2	11–20 m^3 at $30/$m^3$.
Block 3	21–30 m^3 at $35/$m^3$.
Block 4	31–40 m^3 at $42/$m^3$.
Block 5	41+ m^3 at $50/$m^3$.
Environmental fee	10% of basic charge.
Meter maintenance	$15 per bill for 1/2-inch meter.
VAT	12% of basic charge + fees.
Late payment penalty	10% of overdue amount, applied once, 3 days after due date.

5 Required billing acceptance totals

Case	Expected total
37 m^3 example with $500 arrears	$2,037.81
6 m^3, no arrears	$324.80
55 m^3, no arrears	$2,567.04

The roadmap explicitly requires these three cases to match exactly in automated tests.

Target Architecture

The roadmap's target architecture contains the Next.js web application, API Gateway, Auth, Customer, Reading, Billing, Payment and Notification services, with separate databases, RabbitMQ, Redis, MinIO/S3, Mailpit, and observability tooling.

Service ownership matrix

Component	Owns	Database
Gateway	Routing, JWT pre-check, rate limiting, request IDs, CORS, timeouts	None
Auth	Users, roles, refresh sessions, login events, staff/customer identity links	auth_db
Customer	Customers, accounts, addresses, zones, meters, meter installations/history, routes	customer_db
Reading	Billing periods, route assignments, readings, validation state, review decisions, reading photos metadata	reading_db
Billing	Tariffs, tariff versions, bill runs, bills, bill lines, adjustments, ledger, billing read model	billing_db
Payment	Payments, checkout attempts, provider references, webhook events, receipts	payment_db
Notification	Notification preferences, templates, delivery records, notification status	notification_db
Web	UI state and BFF session/cookie handling; no authoritative domain ownership	None

Synchronous versus asynchronous boundaries

Interaction	Mechanism	Reason
Browser -> Web	HTTPS	Secure BFF/session boundary.
Web -> Gateway	HTTPS	Single public API entry point.
Gateway -> Services	HTTP	Request/response operations.
Auth token verification	Local JWT/JWKS verification	Avoid calling Auth for every request.
Reading -> Customer data	HTTP initially; later read model/event strategy	Roadmap explicitly requires an ADR and revisiting it.
Reading -> Billing	Events	Billing eventually builds its own consumption read model.
Billing -> Payment	Events/API as appropriate	Payment state is asynchronous and idempotent.
Billing -> Notification	Events	bill.issued and related events.
Payment -> Billing	payment.received event	Ledger credit must be idempotent.
All event publication	Transactional outbox	Prevents DB/event loss window.
Event consumers	Inbox/idempotency record	Prevents duplicate side effects.

Repository structure

```
aquaflow/
├─ apps/
│  └─ web/
├─ services/
│  ├─ gateway/
│  ├─ auth/
│  ├─ customer/
│  ├─ reading/
│  ├─ billing/
│  ├─ payment/
│  └─ notification/
├─ packages/
│  ├─ shared-config/
│  ├─ logger/
│  ├─ auth-middleware/
│  └─ contracts/
├─ infra/
│  ├─ compose/
│  ├─ k8s/
│  ├─ terraform/
│  └─ observability/
├─ docs/
│  ├─ adr/
│  ├─ runbooks/
│  ├─ journal.md
│  ├─ glossary.md
│  ├─ architecture.md
│  ├─ api/
│  └─ diagrams/
├─ tests/
│  └─ e2e/
└─ .github/workflows/
```

The source roadmap gives essentially this structure and warns not to share domain logic or database models between services.

Technology Choices

Use these technologies as the starting point. If a major technology choice needs to be changed, record the reason in an Architecture Decision Record (ADR).

Area	Decision for this implementation plan	Why
Node.js	Current LTS + TypeScript strict	Matches roadmap and provides one language across services/web.
HTTP	Fastify	Recommended roadmap option; lightweight and suitable for typed APIs.
ORM	Prisma	Good migration/schema workflow for a learning project; raw SQL remains available for optimized queries.
Validation	Zod	Explicit roadmap recommendation; useful for request and event schemas.
Messaging	RabbitMQ	Roadmap recommendation and appropriate for queues/DLQ.
Frontend	Next.js App Router + React	Explicit roadmap target.
Styling	Tailwind CSS + shadcn/ui	Concrete implementation choice; replaceable through ADR.
Unit/integration test	Vitest + Supertest + Testcontainers	Matches roadmap's testing options.
E2E	Playwright	Explicit roadmap recommendation.
Contract testing	Pact	Explicit roadmap option for service relationships.
Load testing	k6	Explicit capstone requirement.
Containers	Docker + Docker Compose	Roadmap target.
Kubernetes	kind locally	Keeps capstone reproducible without cloud cost.
K8s packaging	Helm	Concrete choice; reusable service chart is suitable.
GitOps	Argo CD	Roadmap recommendation.
Registry	GHCR	Roadmap target.
IaC	Terraform optional	Use only when cloud deployment is needed.
Observability	OpenTelemetry + Prometheus + Grafana + Loki + Tempo	Matches roadmap target.
Local email	Mailpit	Explicit roadmap requirement.
Object storage	MinIO	Explicit roadmap requirement.

Developer Tools Required

- Git and GitHub
- Node.js current LTS and npm/pnpm
- Docker Desktop or Docker Engine + Compose
- VS Code or equivalent TypeScript IDE
- PostgreSQL client such as pgAdmin or Adminer
- kubectl and kind
- Helm
- GitHub Actions access and GHCR package access
- Optional: Terraform for cloud
- Optional: an API client such as Bruno/Postman/Insomnia
- A diagramming tool such as draw.io or Excalidraw

Standard Local Development Commands

```
make up
make down
make logs
make seed
make test
make test-e2e
make lint
make typecheck
make reset
make cluster-up
make cluster-down
make deploy
make status
```

Coding and API Standards

API rules

- Base path: /api/v1.
- Use plural nouns and kebab-case: /api/v1/billing-periods/:id/bill-runs.
- JSON fields use camelCase.
- Resource IDs are UUIDs; UUID v7 is preferred where supported.
- Human-readable account numbers and bill numbers are separate fields from UUID IDs.
- Timestamps use ISO-8601 UTC. Date-only values use YYYY-MM-DD.
- Money uses integer minor units plus currency, for example { amount: 203781, currency: 'USD' }.
- Use cursor pagination for large collections.
- Return a consistent error object containing code, message, optional field details, and requestId.
- Use 401 for unauthenticated, 403 for authenticated-but-forbidden, 404 for missing or intentionally hidden resources, 409 for state conflicts, 422 for semantic validation, 429 for rate limiting, and 503 for unavailable dependencies.
- Use Idempotency-Key on retryable mutation endpoints where duplicate effects would be harmful.

Error contract

```
{
  "error": {
    "code": "METER_ALREADY_INSTALLED",
    "message": "Account already has an active meter.",
    "details": [
      {"field": "meterSerial", "issue": "must be replaced, not added"}
    ],
    "requestId": "req-..."
  }
}
```

Service internal layering

```
src/
├─ routes/
├─ domain/
├─ repositories/
├─ events/
├─ config.ts
└─ server.ts

Typical request flow:
route/controller -> validation -> domain/service -> repository -> database
                                        └-> event/outbox when required
```

Configuration rules

- All environment variables are validated at startup.
- No secrets have fallback production values.
- Provide .env.example with variable names but no real credentials.
- Development-only credentials may be documented locally, never production credentials.
- Separate environment configuration from code.

5 Logging rules

- Structured JSON logs.
- Every request receives/propagates x-request-id.
- Trace and correlation IDs are included where available.
- Never log passwords, access tokens, refresh tokens, payment secrets, or sensitive personal data.
- Use consistent fields: timestamp, level, service, requestId, traceId, route, statusCode, durationMs, errorCode.

Complete Development Schedule by Milestone

M0 — Orientation and domain discovery

Main tasks:

1. Read and understand the domain primer.
2. Create docs/glossary.md with at least 15 terms.
3. Write 12+ user stories covering Admin, Billing Officer, Meter Reader and Customer.
4. Create context map showing service ownership and sync/async relationships.
5. Create billing spreadsheet reproducing the three expected totals.
6. Document meter replacement and duplicate-payment edge cases.
7. Create ADR-0001 for microservice architecture.
8. Create Git repository, README and development journal.
9. List assumptions and unresolved business questions.

Completion check: Checkpoint A: domain understanding, context map and service boundaries.

M1 — Monorepo and Customer Service

Main tasks:

10. Initialize workspace, strict TypeScript, ESLint, Prettier and shared configuration.
11. Create PostgreSQL Compose setup.
12. Create Customer Service with config validation.
13. Model customers, accounts, service addresses, zones, meters and meter installations.
14. Implement customer/account/meter endpoints.
15. Implement account state transitions.
16. Enforce one active meter per account.
17. Implement meter installation and replacement history.
18. Add Zod validation and global error handling.
19. Add structured logging and request IDs.
20. Create seed data: 3 zones, 60 customers, 75 accounts, 80 meters.
21. Add unit and integration tests.
22. Expose OpenAPI docs and health/readiness endpoints.

Completion check: Checkpoint: clean clone can start service and tests pass.

M2 — Auth and RBAC

Main tasks:

23. Create Auth Service and auth_db.
24. Model users, roles, refresh tokens and login events.
25. Implement login, refresh, logout, /me and staff user management.
26. Implement customer self-registration through account verification.
27. Generate RSA/EC signing keys.
28. Expose JWKS endpoint.
29. Build shared JWT verification/permission middleware.
30. Apply permission matrix to Customer Service.
31. Implement refresh-token rotation and reuse detection.
32. Add failed-login rate limiting/lockout.
33. Add security audit events.
34. Seed one local user per role.
35. Write permission-matrix tests for role x endpoint.

Completion check: Checkpoint B: code structure, testing and security basics.

M3 — Gateway and containerization

Main tasks:

36. Create multi-stage Dockerfiles for services.
37. Create .dockerignore files.
38. Expand Compose to all existing services plus Redis.
39. Add health checks and service startup dependencies.
40. Decide and document migration startup strategy.
41. Implement gateway routing.
42. Implement JWT pre-check, request IDs, CORS, rate limiting and timeouts.
43. Ensure only gateway publishes application ports.
44. Run containers as non-root.
45. Add root operational scripts/Makefile.
46. Update architecture documentation.

Completion check: Fresh clone starts healthy; gateway returns controlled 503 when dependency fails.

M4 — Meter Reading Service

Main tasks:

47. Add MinIO and Reading Service database.
48. Model billing periods, reading windows, routes and assignments.
49. Implement reading states: SUBMITTED, VALIDATED, FLAGGED, APPROVED, REJECTED, ESTIMATED.
50. Implement single and batch submission.
51. Implement Idempotency-Key handling.
52. Implement direct-to-MinIO photo upload using pre-signed URLs.
53. Implement R1–R7 as independent rule functions.
54. Build review queue and approval/override/rejection flow.
55. Implement estimated reads.
56. Implement meter rollover handling.
57. Implement meter replacement consumption formula.
58. Expose consumption endpoint for Billing.
59. Add permission tests and route-assignment enforcement.

Completion check: Checkpoint is after M5, but M4 must be complete before billing.

M5 — Billing Engine

Main tasks:

60. Create Billing Service database.
61. Model tariffs, versions, tiers, bill runs, bills, bill lines, ledger entries and adjustments.
62. Seed Residential, Commercial and Industrial tariffs.
63. Implement pure charge calculator.
64. Use exact money representation and documented rounding.
65. Implement bill-run preview -> review -> issue.
66. Carry arrears forward.
67. Apply late penalties through a safe scheduled/background job.
68. Implement immutable issued bills.
69. Implement adjustments/credit/debit memos with reason and approver.
70. Generate PDFs and store in MinIO.
71. Expose ledger and PDF endpoints.
72. Store source reading IDs and tariff version on every bill.
73. Make bill runs idempotent.
74. Test the three mandatory billing totals.
75. Test 1,000-account local bill run target.

Completion check: Checkpoint C: billing correctness, data model and ADR quality.

M6 — Next.js staff portal

Main tasks:

76. Create apps/web.
77. Implement secure login/logout using httpOnly cookies.
78. Implement silent refresh and route protection.
79. Build role-aware navigation and application shell.
80. Build customers/accounts list and detail views.
81. Add server-side search, filtering and pagination.
82. Build meter history view.
83. Build flagged-reading review queue with photo and previous-reading chart.
84. Build approve/override/reject workflow.
85. Build tariff management.

86. Build billing-period and bill-run wizard.
87. Build dashboard KPIs.
88. Build users/roles admin screen.
89. Build audit-log screen.
90. Add loading, empty and error states.
91. Write Playwright tests.

Completion check: Staff can operate the core workflow securely.

M7 — Events, payments and notifications

Main tasks:

92. Add RabbitMQ and Mailpit.
93. Create typed event contracts.
94. Implement transactional outbox in Reading and Billing.
95. Create event relay/publisher.
96. Move Billing to event-fed consumption read model.
97. Build Payment Service.
98. Implement OTC payments.
99. Implement online/mock checkout.
100. Implement signed webhook handling.
101. Implement payment idempotency.
102. Emit payment.received.
103. Credit Billing ledger exactly once.
104. Handle overpayments as credit balance.
105. Build Notification Service.
106. Consume bill.issued, payment.received, bill.overdue and reading.flagged.
107. Render templates and record delivery.
108. Implement retries with exponential backoff and DLQ.
109. Implement inbox/processed-event tracking.
110. Add DLQ inspect/replay tool.

Completion check: Checkpoint D: distributed systems, failure modes, idempotency and consistency.

M8 — Customer portal and reader PWA

Main tasks:

111. Build customer bill list and bill detail/PDF view.
112. Build usage history/chart.
113. Build online payment flow.
114. Show pending -> confirmed payment state.
115. Build notification preferences.
116. Build reader route screen.
117. Build offline local queue.
118. Store photos locally until sync.
119. Implement background/manual sync.
120. Use the idempotent batch API.
121. Handle conflicts and partial sync results.
122. Test airplane-mode route completion.
123. Test 360px/mobile layouts.
124. Test IDOR/account-isolation cases.

Completion check: Milestone may be reduced to reader PWA only if necessary, per roadmap.

M9 — Quality gates and

Main tasks:

125.Create PR workflow.
126.Run lint, typecheck, unit and integration tests for affected services.
127.Add Pact contract tests for at least two service relationships.
128.Add full E2E bill-life test using Compose.
129.Build/push changed images to GHCR.
130.Tag images by Git SHA; use semver on releases; never deploy latest.
131.Add dependency audit.
132.Add Trivy/Grype image scanning.
133.Add secret scanning.
134.Enable branch protection.
135.Add Dependabot or Renovate.
136.Automate changelogs/releases.
137.Use affected-service detection/caching.

Completion check: Every PR is protected by automated quality checks.

M10 — Kubernetes and IaC

Main tasks:

138.Script kind cluster creation.
139.Install ingress controller and local registry.
140.Create Helm charts/manifests for every service and web app.
141.Deploy PostgreSQL, RabbitMQ, Redis and MinIO.
142.Configure ingress for app and API.
143.Implement Secrets/ConfigMaps and environment overlays.
144.Add migration Jobs/init strategy.
145.Add resource requests/limits.
146.Add readiness/liveness probes.
147.Add HPA to Reading Service and gateway.
148.Add PodDisruptionBudgets.
149.Install/configure Argo CD.
150.Connect Git repository to GitOps deployment.
151.Practice rolling update, rollback, scaling and node drain.
152.Optional: create Terraform cloud environment.

Completion check: Checkpoint E: infrastructure, deployment workflow and secrets.

M11 — Observability and resilience

Main tasks:

153.Instrument services with OpenTelemetry.
154.Propagate trace context across HTTP and RabbitMQ.
155.Collect metrics with Prometheus.
156.Centralize logs with Loki.
157.Collect traces with Tempo/Jaeger.
158.Build Grafana dashboards.
159.Define SLIs/SLOs.
160.Create alerts for availability, latency, error rate, queue backlog and failed jobs.
161.Add timeouts, retries, circuit breakers where suitable.
162.Test dependency failures.
163.Run game-day experiments.

164.Document findings and fixes.
165.Ensure payment trace can cross services.

Completion check: Observability must make failures diagnosable within minutes.

M12 — Hardening, performance and capstone

Main tasks:

166.Perform OWASP Top 10 review.
167.Automate authorization/IDOR tests.
168.Add security headers and CSP.
169.Add request/body size limits.
170.Rotate secrets and JWT signing key using runbooks.
171.Create k6 load tests for reading sync, bill run and portal browsing.
172.Load target: 10,000 accounts, 50 concurrent readers syncing, 200 concurrent customer users.
173.Identify and fix top three bottlenecks.
174.Automate PostgreSQL backups.
175.Perform restore drill into a fresh cluster.
176.Finish README, architecture docs, ADR index, API docs, runbooks and onboarding guide.
177.Write retrospective.
178.Prepare and deliver capstone presentation.

Completion check: Final checkpoint F: capstone presentation and rubric assessment.

Detailed Backend Development Work

Customer Service

Feature	Implementation requirements	Tests
Customers	Create, retrieve, search; validate contact/address data.	Create, duplicate handling, search, invalid input.
Accounts	Create, list, filter by zone/status, status transitions.	Allowed/forbidden transitions, pagination.
Meters	Register inventory, serial uniqueness, size, register/max digits.	Duplicate serial, numeric bounds.
Installation	Install meter with initial reading; only one active meter.	Second active meter returns 409.
Replacement	Close old installation, record final reading, create new installation/initial reading.	Historical date lookup; consumption across replacement.
Health	/healthz and /readyz.	DB unavailable affects readiness, not process liveness.

Auth Service

- Password hashing with Argon2 or bcrypt; recommended: Argon2id.
- Access tokens signed asymmetrically.
- JWKS endpoint supports key IDs (kid) and rotation.
- Refresh tokens are rotated and single-use.
- Reuse of an old refresh token revokes the affected session.
- Permission checks happen server-side.
- Customer self-registration requires account verification.
- Failed-login events and security actions are auditable.

Reading Service

- Billing periods are zone-aware and have explicit reading windows.
- Route assignments are main source of truth for reader authorization.
- Reading submissions support both individual and batch payloads.
- Photos use pre-signed object-storage URLs.
- Validation rules are independent pure functions.
- Review decisions record actor, time, previous value, new value and reason.
- Estimated readings are clearly marked.
- Closing a period is blocked while unresolved flagged readings exist unless an explicit override is recorded.
- Consumption endpoint returns final approved consumption suitable for billing.

Billing Service

- Tariffs are effective-dated and immutable after use.
- Charge calculation is a pure function with no database or network access.
- Issued bills contain the tariff version and source reading IDs.
- Ledger is the source of truth for balance.
- Bills cannot be edited after issue.
- Adjustments create new ledger effects.
- Bill run can safely retry without duplicate bills.
- Penalty application is once-only per overdue bill/charge.
- PDF output is repeatable from stored bill data.

Payment Service

- Payment status model should include at minimum PENDING, CONFIRMED, FAILED and optionally REFUNDED.
- Online checkout uses a provider/mock abstraction.
- Webhook authenticity is verified.
- Webhook event IDs are deduplicated.
- Idempotency key is unique per payment attempt.
- Payment confirmation publishes payment.received.
- Receipt is generated/stored after confirmation.
- Overpayments become account credit rather than disappearing.

Notification Service

- Store customer notification preferences.
- Templates are versioned or otherwise traceable.
- Delivery records store event ID, channel, recipient reference, status, attempt count and timestamps.
- Local email uses Mailpit.
- Failed messages retry with exponential backoff.
- Poison messages move to DLQ after configured retry count.
- DLQ messages can be inspected and replayed safely.

Database Design Plan

The roadmap requires one logical PostgreSQL database per service. Exact columns can be refined during implementation, but the following entity inventory should be used as the starting database contract.

auth_db

- users: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- roles: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- user_roles: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- refresh_tokens: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- login_events: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- customer_account_links: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

customer_db

- customers: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- service_addresses: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- zones: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- routes: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- route_assignments: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- accounts: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- meters: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- meter_installations: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

reading_db

- billing_periods: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- reading_assignments: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- readings: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- reading_flags: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- reading_reviews: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- reading_photos: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- idempotency_keys: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

- outbox_events: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

billing_db

- tariffs: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- tariff_versions: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- tariff_tiers: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- tariff_fees: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- bill_runs: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- bills: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- bill_lines: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- ledger_entries: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- adjustments: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- consumption_read_model: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- outbox_events: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- idempotency_keys: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

payment_db

- payments: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- payment_attempts: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- provider_webhooks: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- receipts: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- idempotency_keys: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- outbox_events: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

payment_db

- payments: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- payment_attempts: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- provider_webhooks: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- receipts: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

- idempotency_keys: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.
- outbox_events: define primary key, timestamps, status/state fields where needed, foreign IDs, uniqueness constraints, and indexes based on query patterns.

Critical database constraints

- Meter serial numbers must be unique.
- An account cannot have more than one active meter installation at the same time.
- A meter cannot have overlapping active installations.
- A reading must be unique for meter + billing period.
- Idempotency keys must be scoped to operation/client and stored with request fingerprint/result.
- Issued bill number must be unique.
- One bill per account + billing period unless the domain explicitly supports rebills through a separate correction mechanism.
- Ledger entries are append-only.
- Tariff version identifiers are immutable once referenced by an issued bill.
- Event IDs are unique in outbox/inbox processing.

Required indexes

- Customer search fields used by the staff portal.
- Account zone/status and account number.
- Meter serial and active installation lookup.
- Readings by meter + period and route + period + status.
- Bills by account + period + status + due date.
- Ledger by account + occurredAt.
- Payments by provider reference, idempotency key and account.
- Notification deliveries by status and next retry time.
- Outbox by published status and createdAt.
- Inbox by consumer + event ID.

Frontend and User Interface Plan

Staff portal routes

Route	Purpose	Required role
/login	Authentication	All staff
/dashboard	KPIs and operational overview	Admin/Billing officer
/customers	Search/list customers	Admin/Billing officer
/customers/:id	Customer detail	Admin/Billing officer
/accounts/:id	Account, meter, readings, bills, ledger	Admin/Billing officer; customer own account through customer area
/readings/review	Flagged reading queue	Admin/Billing officer
/tariffs	Tariff management	Admin
/billing/periods	Billing period management	Admin/Billing officer
/billing/runs/:id	Preview/review/issue	Admin/Billing officer
/payments	Payment status and records	Admin/Billing officer

Route	Purpose	Required role
/users	Staff users/roles	Admin
/audit	Sensitive action audit log	Admin

Customer portal routes

- /customer/login
- /customer/dashboard
- /customer/bills
- /customer/bills/:id
- /customer/usage
- /customer/payment/:id
- /customer/settings/notifications

Reader PWA routes

- /reader/login
- /reader/route
- /reader/route/:routeId
- /reader/stop/:accountId
- /reader/sync
- /reader/settings

UX requirements

- Responsive from 360px width.
- Keyboard accessible staff portal.
- Lighthouse accessibility target >= 90 on main staff pages.
- Every page has loading, empty, error and success states.
- Reader app clearly displays offline/online state and queued records.
- Customer payment state visibly distinguishes pending from confirmed.
- Sensitive staff actions require confirmation and display reason/actor where suitable.
- Do not expose access tokens to browser JavaScript; use secure httpOnly cookies.

Event-Driven Contract Plan

The roadmap supplies an event envelope with id, type, version, occurredAt, source, correlationId and data. Event names are past-tense facts.

Event naming

```
<context>.<entity>.<past-tense-verb>

Examples:
reading.reading.submitted
reading.reading.validated
reading.reading.flagged
reading.reading.approved
reading.reading.rejected
billing.bill.issued
billing.bill.overdue
payment.payment.received
notification.notification.sent
```

The exact final catalog should be maintained in packages/contracts and docs/event-catalog.md. Avoid putting full database entities into events.

Event envelope

```
{
  "id": "uuid",
  "type": "billing.bill.issued",
  "version": 1,
  "occurredAt": "2026-10-05T08:30:00Z",
  "source": "billing-service",
  "correlationId": "req-...",
  "data": {
    "billId": "...",
    "accountId": "...",
    "amountDue": 203781,
    "dueDate": "2026-10-20"
  }
}
```

Reliability requirements

- Write business change and outbox event in one database transaction.
- Relay unpublished events to RabbitMQ.
- Consumers acknowledge only after successful processing.
- Consumers record event IDs before/with side effects according to a within the same database transaction safe strategy.
- Duplicate events must not produce duplicate ledger entries, notifications or payments.
- Use exponential backoff.
- Poison messages move to DLQ after N attempts.
- DLQ replay must preserve idempotency.
- Correlation ID and trace context must be propagated.

Complete Testing Plan

Testing is divided into several levels so that errors can be found early and the complete business process can be checked.

Test pyramid

Layer	Scope	Required examples
Unit	Pure domain/business functions	Tariff calculator; validation rules; state transitions; permission decisions.
Integration	Service + real PostgreSQL/dependencies	Repositories, transactions, migrations, API endpoints.
Contract	Service-to-service API/event compatibility	Web -> gateway -> billing; billing -> reading event contract.
E2E	Complete user/business journeys	Life of a bill from offline reading to payment.
Security	Authorization and input abuse	IDOR, role escalation, invalid JWT, replay, webhook signature.
Load	Performance at target load	Reading sync, bill run, portal browsing.
Resilience	Failure/restart scenarios	Service outage, broker outage, duplicate events, pod deletion.

Layer	Scope	Required examples
Backup/restore	Operational recovery	Restore database into fresh environment.

Mandatory business test cases

- 37 m^3 residential bill with $500 arrears = $2,037.81.
- 6 m^3 bill = $324.80.
- 55 m^3 bill = $2,567.04.
- Negative reading is flagged unless rollover/replacement explains it.
- High usage >300% of three-period average is flagged.
- Two consecutive zero-usage periods on active account are flagged.
- Reading outside period window is rejected.
- Duplicate reading submission returns the original result.
- No-access creates estimated reading.
- Flagged reading without photo warns reader.
- Meter replacement consumption formula works.
- Second active meter installation returns 409.
- Issued bill cannot be mutated.
- Tariff change only affects periods on/after effective date.
- Duplicate payment event credits ledger once.
- Duplicate webhook does not create a second payment.
- Customer cannot access another customer's account.
- Reader cannot submit outside assigned route.

Final E2E scenario

179. Reader logs in.
180. Reader downloads/opens assigned route.
181. Device goes offline.
182. Reader completes multiple stops, enters readings, records no-access where applicable, and captures photos.
183. Reader reconnects.
184. Queued readings sync through the batch endpoint.
185. System validates readings.
186. Flagged readings appear in staff review queue.
187. Billing officer approves/overrides/rejects as required.
188. Billing period is closed.
189. Bill run previews totals and exceptions.
190. Bill run issues bills.
191. PDF is stored.
192. bill.issued is published through the outbox.
193. Notification email appears in Mailpit.
194. Customer opens portal and sees bill.
195. Customer starts online/mock payment.
196. Payment is pending.
197. Webhook confirms payment.
198. payment.received is consumed exactly once.
199. Ledger is credited.
200. Customer sees payment confirmed and updated balance.

CI/CD and Release Plan

1 Pull request pipeline

201. Checkout repository.
202. Install dependencies using lockfile.
203. Restore package/build cache.
204. Lint affected packages/services.
205. Type-check affected packages/services.
206. Run unit tests.
207. Start required integration dependencies.
208. Run integration tests.
209. Run contract tests.
210. Run security/dependency checks.
211. Build affected services.
212. Fail the PR on any required check failure.

Main/release pipeline

213. Run full quality gates.
214. Build changed Docker images.
215. Scan images.
216. Generate SBOM if enabled.
217. Tag images with Git SHA.
218. Push images to GHCR.
219. Create release/semver tag when suitable.
220. Update GitOps image references.
221. Argo CD synchronizes cluster.
222. Run post-deployment smoke/E2E checks.

Required workflow files

```
.github/workflows/
├─ pull-request.yml
├─ e2e.yml
├─ build-images.yml
├─ security.yml
├─ release.yml
└─ deploy.yml
```

Branch and commit policy

- Branch per task.
- Pull request per task, even for solo development.
- Conventional Commits, e.g. feat(billing): add tier calculator.
- Main must remain deployable.
- Require successful CI checks before merge.
- Do not commit .env files or secrets.

Kubernetes Deployment Plan

Cluster components

- kind cluster.
- Ingress controller.
- Local image registry.
- AquaFlow namespace(s).
- Argo CD namespace.
- PostgreSQL, RabbitMQ, Redis and MinIO deployments/stateful components.
- All AquaFlow services and web app.
- ConfigMaps and secret references.
- Persistent volumes for stateful dependencies.

Required workload controls

- Readiness and liveness probes.
- CPU/memory requests and limits.
- Rolling deployment strategy.
- HPA for gateway and Reading Service.
- PodDisruptionBudget where suitable.
- Migration Job before new application version receives traffic.
- Non-root security context.
- No manual kubectl edits as the source of desired configuration.

3 GitOps flow

```
Developer -> Pull Request -> CI checks
        -> merge main -> image build -> GHCR
        -> GitOps manifest/image update
        -> Argo CD detects Git change
        -> Kubernetes rollout
        -> health checks
        -> smoke test
```

4 Cloud is optional

The supplied roadmap explicitly allows local Kubernetes to satisfy the capstone and treats cloud deployment as optional. Therefore, do not introduce cloud costs until the local Kubernetes implementation is stable. If cloud is later selected, use Terraform and document the provider, cost controls, remote state and destruction procedure.

Monitoring and System Reliability Plan

Logs

- JSON logs from every service.
- Centralize in Loki.
- Include service, environment, requestId, traceId, spanId, eventId and errorCode where applicable.
- Create dashboards for errors, latency and event processing.

Metrics

Metric category	Examples
HTTP	request count, error count, latency p50/p95/p99
Database	query latency, connection pool usage, errors
RabbitMQ	queue depth, publish failures, consumer failures, retry count, DLQ count
Billing	bill runs started/completed/failed, bills issued, bill-run duration
Reading	readings submitted, flagged, estimated, approved, rejected
Payments	pending, confirmed, failed, duplicate webhook attempts
Notifications	sent, failed, retried, DLQ
Infrastructure	CPU, memory, pod restarts, readiness failures

Distributed traces

The final system should be able to follow a user action through gateway -> service -> broker -> consumer -> ledger/notification. The roadmap specifically requires a single distributed trace that can follow a payment across services.

SLO starting point

Define explicit targets during M11. At minimum define availability, API latency, error rate, event processing delay and bill-run completion. Store the selected thresholds in docs/slo.md and configure Grafana alerts from those values.

Resilience experiments

- Stop Customer Service and verify gateway returns controlled failure.
- Stop Reading Service during an approved-reading workflow and verify documented behavior.
- Kill a service after DB commit and before event publish; outbox must recover the event.
- Deliver the same event twice; side effect must occur once.
- Publish malformed event; it must eventually reach DLQ without blocking healthy messages.
- Delete a Kubernetes pod; replacement must become ready.
- Perform rolling Billing Service update during k6 smoke test.
- Restore database backup into a fresh environment.

Documentation and Project Files

Required repository documentation

Path	Purpose
README.md	Project overview, prerequisites, setup, common commands, architecture entry point.
docs/architecture.md	Current architecture and service boundaries.
docs/glossary.md	Domain and technical vocabulary.
docs/user-stories.md	User stories and acceptance criteria.
docs/journal.md	3–5 lines per work session.

Path	Purpose
docs/assumptions.md	Business assumptions and unresolved questions.
docs/event-catalog.md	Event names, versions, publishers, consumers and schemas.
docs/api/	Generated/manual API documentation.
docs/slo.md	SLIs, SLOs, alert thresholds.
docs/runbooks/	Operational procedures.
docs/adr/	Architecture Decision Records.
docs/security-review.md	Final security review.
docs/load-test-results.md	Performance test methodology and results.
docs/retrospective.md	Final reflection and future improvements.

ADR plan

ADR	Decision
ADR-0001	Adopt microservice architecture.
ADR-0002	Select HTTP framework.
ADR-0003	Select ORM/data access strategy.
ADR-0004	Select authentication/token strategy.
ADR-0005	Customer-account identity linking.
ADR-0006	Gateway implementation.
ADR-0007	Reading Service data ownership strategy.
ADR-0008	Money representation and rounding.
ADR-0009	Bill-run execution strategy.
ADR-0010	PDF generation approach.
ADR-0011	Broker selection and topology.
ADR-0012	Event contract/versioning strategy.
ADR-0013	BFF versus direct browser-to-gateway.
ADR-0014	Reader PWA storage/sync approach.
ADR-0015	Kubernetes Helm versus Kustomize.
ADR-0016	GitOps/deployment strategy.
ADR-0017	Observability stack.
ADR-0018	Backup and restore strategy.
ADR-0019	Security baseline.
ADR-0020	A significant decision later reversed.

The roadmap expects roughly 12–20 ADRs and explicitly requires at least one decision that is later reversed as understanding grows.

Development Task Format

For actual software development, create GitHub Issues from the following pattern. Each task should be independently testable and should identify its service, acceptance criteria and dependencies.

Issue ID pattern	Example	Required contents
DOM-###	DOM-001 Define account lifecycle	Context, rule, acceptance criteria, related ADR.
CUST-###	CUST-012 Add meter replacement	API, DB, tests, Definition of Done.
AUTH-###	AUTH-005 Implement refresh rotation	Security requirements and tests.
READ-###	READ-018 Implement HIGH_USAGE rule	Rule formula, pass/fail tests.
BILL-###	BILL-021 Implement tier calculator	Inputs, outputs, mandatory billing tests.
WEB-###	WEB-015 Build review queue	UI states, permission behavior, Playwright test.
PAY-###	PAY-008 Verify webhook signature	Provider contract, replay test.
NOTIF-###	NOTIF-006 Add DLQ replay	Operational safety and idempotency.
PWA-###	PWA-010 Offline batch sync	Offline behavior and duplicate test.
CI-###	CI-004 Add image scanning	Critical finding must fail.
K8S-###	K8S-013 Deploy Billing via Helm	Probe, resources, rollout/rollback test.
OBS-###	OBS-009 Trace payment flow	Trace must span required services.
SEC-###	SEC-005 Add IDOR suite	Cross-account access attempts.
PERF-###	PERF-003 Load bill run	Target, metrics and result.
OPS-###	OPS-002 Restore PostgreSQL backup	Measured recovery time.

Definition of Done for Each Task

- Code implemented.
- Automated tests added/updated.
- Validation and authorization covered.
- Logs/metrics/traces added where relevant.
- Documentation updated.
- No secrets introduced.
- Lint and typecheck pass.
- Relevant integration/E2E tests pass.
- PR reviewed/approved.
- Demoable behavior exists.

Final Project Acceptance Checklist

The project is complete only when the required functions, architecture, infrastructure, security, and documentation have been demonstrated.

The following is the final gate. The roadmap states that the final capstone must show the complete bill life on Kubernetes and satisfy the functional, architecture/code, and infrastructure/operations criteria.

Functional

- Offline reader submission works.
- Validation and anomaly review work.
- Bill run works.
- PDF bill is generated and stored.
- Bill-issued notification is delivered.
- Customer can pay online through the mock/sandbox flow.
- Payment updates ledger exactly once.
- All four roles behave according to permission matrix.
- Meter replacement is demonstrable.
- Estimated reading is demonstrable.
- Arrears are demonstrable.
- Penalty is demonstrable.
- Adjustments are demonstrable.
- Three mandatory billing totals pass automated tests.

Architecture and code

- Six backend services plus gateway exist.
- Each service has its own database.
- No service reads another service database.
- Services are independently buildable/deployable.
- Outbox is implemented.
- Consumers are idempotent.
- Duplicate/replayed events produce no duplicate effects.
- At least 12 ADRs exist.
- At least one architectural decision was later reversed and documented.

Infrastructure and operations

- One command starts the full local Compose stack.
- One command creates the local Kubernetes environment.
- CI blocks broken changes.
- CD delivers merged changes automatically.
- No plaintext secrets in Git.
- Images are scanned.
- Images run non-root.
- Resource limits exist.
- Distributed payment trace works.
- SLO dashboards and alerts exist.
- Backup restore has been successfully performed.

- Runbook documents restore procedure.

Final usability test

A developer who has never seen the repository should be able to follow README.md and run the project locally in under 30 minutes. This is a final documentation acceptance criterion from the roadmap.

Recommended Development Order

This order is recommended to reduce rework and keep the project manageable.

Do not attempt to build all services simultaneously. The safest sequence is to finish one vertical slice at a time while preserving the final architecture.

223. Set up Git repository, project board, journal, ADR system and baseline tooling.
224. Complete domain model, user stories, business calculations and context map.
225. Build Customer Service and database.
226. Build Auth and secure Customer Service.
227. Add gateway and containerization.
228. Build Reading Service and prove all reading rules.
229. Build Billing calculator independently before wiring the service.
230. Integrate Reading -> Billing and prove the complete bill calculation.
231. Build staff portal around the completed backend workflow.
232. Add RabbitMQ/outbox/inbox before adding payment/notification complexity.
233. Build Payment Service and exactly-once ledger effect.
234. Build Notification Service and DLQ tooling.
235. Build customer portal and reader PWA.
236. Automate CI/CD before Kubernetes so the deployment system consumes known-good images.
237. Move the entire stack to local Kubernetes.
238. Add observability and resilience testing.
239. Perform security, performance and backup/restore work.
240. Run the final capstone scenario repeatedly until it is deterministic.
241. Freeze release candidate, document known limitations, and present.

1 Important Things to Avoid

- Do not start with Kubernetes before the application works in Compose.
- Do not build the UI before the backend contracts are stable enough to use.
- Do not let services share database tables.
- Do not put tariff values directly into TypeScript code.
- Do not use floating point for money.
- Do not edit issued bills.
- Do not publish broker events without an outbox.
- Do not assume a message is delivered exactly once.
- Do not rely on frontend authorization.
- Do not add cloud infrastructure before local deployment is repeatable.
- Do not skip tests to catch up; skip stretch goals instead.

Specification for Software and AI-Assisted Development

This section gives a standard format for assigning development tasks to a coding assistant or another developer.

This section is intentionally structured so the document can be used as a project specification when working with coding assistants, IDE agents, code-generation tools, or another developer. It should be treated as a specification source, not as a request to generate the entire system in one step.

1 Instruction hierarchy for implementation

242. Follow the business rules and acceptance criteria in this document.
243. Follow service ownership boundaries.
244. Follow API/event contracts.
245. Follow security requirements.
246. Follow tests and Definition-of-Done criteria.
247. If implementation conflicts with a documented ADR, follow the ADR unless it is being deliberately superseded.
248. If a requirement is ambiguous, create an explicit assumption or ADR rather than silently inventing behavior.

Prompt/task template for coding software

```
TASK ID:
SERVICE:
GOAL:
BACKGROUND:
REQUIREMENTS:
API CONTRACT:
DATABASE CHANGES:
EVENTS:
AUTHORIZATION:
VALIDATION:
ERROR CASES:
TESTS REQUIRED:
DOCUMENTATION:
DEPENDENCIES:
DEFINITION OF DONE:
FILES EXPECTED TO CHANGE:
NON-GOALS:
```

Example machine-friendly task

```
TASK ID: BILL-021
SERVICE: billing
GOAL: Implement pure residential tier calculator.

INPUT:
- consumptionM3: integer >= 0
- tariffVersion: effective tariff data
- meterSize: tariff fee context

OUTPUT:
- ordered bill lines
- basicChargeMinor
- feeMinor
- taxMinor
- currentChargesMinor

RULES:
- minimum charge covers first 10 m3 and is always 25000 minor units
- blocks are cumulative
- environmental fee = 10% of basic charge
- meter maintenance fee = 1500 minor units for 1/2 inch
- VAT = 12% of subtotal before VAT
- rounding = documented half-up strategy

NON-GOALS:
- database access
- PDF generation
- ledger writes

TESTS:
- 6 m3 -> total 32480
- 55 m3 -> total 256704
- 37 m3 example with arrears handled by higher-level billing flow

DEFINITION OF DONE:
- pure function
- no floating-point money
- unit tests pass
- typecheck passes
- documentation updated
```

File naming rules

- Use lowercase kebab-case for docs and configuration files.
- Use service names exactly: gateway, auth, customer, reading, billing, payment, notification.
- Keep shared contracts in packages/contracts.
- Keep domain models inside owning services.
- Keep generated artifacts clearly separated from hand-written source.
- Do not rename established API/event identifiers casually; treat changes as contract changes.

Requirements Traceability Matrix

This section connects each major requirement to the part of the system and the tests that provide evidence.

Requirement group	Primary implementation area	Primary tests/evidence
Customers/accounts/meters	Customer Service	Customer integration tests; DB constraints; API tests
Authentication/RBAC	Auth + auth-middleware	Permission matrix tests; security tests
Gateway	Gateway	Routing, timeout, 401/403/429/503 tests
Reading rules R1–R7	Reading Service	Table-driven rule tests
Offline reading	Reader PWA + Reading batch API	Offline Playwright E2E
Billing correctness	Billing Service	Named billing test cases + property tests
Immutable bills/ledger	Billing Service	Mutation rejection + ledger reconciliation tests
Payments	Payment + Billing	Webhook/idempotency/inbox tests
Notifications	Notification Service	Event consumer + Mailpit tests
Events	RabbitMQ + outbox/inbox	Crash/replay/DLQ tests
Staff portal	Next.js	Playwright + accessibility checks
Customer portal	Next.js	Playwright + authorization tests
CI/CD	GitHub Actions/GHCR	Intentional failing PR checks
Kubernetes	Helm/kind/Argo CD	Cluster deployment and rollout tests
Observability	OTel/Prometheus/Grafana/Loki/Tempo	Trace and dashboard evidence
Security	Gateway/Web/Auth/Webhook	OWASP review + IDOR suite
Performance	k6	Load results
Backup	PostgreSQL + runbook	Successful restore drill

Evidence folder

```
docs/evidence/
├─ m0-domain/
├─ m1-customer/
├─ m2-auth/
├─ m3-gateway/
├─ m4-reading/
├─ m5-billing/
├─ m6-staff-portal/
├─ m7-events-payments/
├─ m8-pwa-customer/
├─ m9-ci/
├─ m10-kubernetes/
├─ m11-observability/
└─ m12-capstone/
```

Store screenshots, test output, benchmark results, demo links, architecture diagrams and mentor feedback here. Do not store secrets, real customer data, payment credentials or private keys.

Master Project Checklist

Use this checklist near the end of the project to confirm that major requirements have been completed.

☐ Repository initialized with branch/commit policy.

☐ Domain glossary completed.

☐ 12+ user stories completed.

☐ Context map completed.

☐ Billing spreadsheet completed.

☐ Assumptions documented.

☐ ADR system established.

☐ Customer Service implemented and tested.

☐ Auth Service and RBAC implemented.

☐ Gateway implemented.

☐ All services containerized.

☐ Reading Service implemented.

☐ All R1–R7 tests passing.

☐ Billing calculator implemented.

☐ Three mandatory billing totals passing.

☐ Ledger and immutable bills implemented.

☐ Staff portal implemented.

☐ RabbitMQ implemented.

☐ Outbox implemented.

☐ Inbox/idempotent consumers implemented.

☐ Payment Service implemented.

☐ Notification Service and DLQ implemented.

☐ Customer portal implemented.

☐ Reader PWA implemented.

☐ Offline 20-stop test passing.

☐ CI/CD implemented.

☐ GHCR images traceable to commits.

☐ Security/image/dependency scanning enabled.

☐ Kubernetes environment repeatable.

☐ Helm/GitOps deployment working.

☐ Secrets are not in Git.

☐ Readiness/liveness/resource controls implemented.

☐ OpenTelemetry traces working.

☐ Prometheus/Grafana/Loki/Tempo operational.

☐ SLOs and alerts defined.

☐ Resilience/game-day tests completed.

☐ OWASP review completed.

☐ IDOR tests completed.

☐ k6 performance targets evaluated.

☐ PostgreSQL backup completed.

☐ Restore drill completed.

☐ README onboarding under 30 minutes.

☐ Runbooks complete.

☐ Retrospective complete.

☐ Capstone E2E scenario passes on Kubernetes.

☐ Capstone presentation ready.

Final Implementation Note

This final section explains the purpose of the development plan and how it relates to the original learning roadmap.

The original roadmap is mainly a learning guide, not a complete software requirements document. This document makes the roadmap easier to implement by defining service responsibilities, recommended technologies, development tasks, API rules, database areas, testing levels, deployment files, and final evidence. It does not silently change the source business rules. Where the source leaves a decision open, the plan labels a recommended implementation choice and requires an ADR so that the decision remains explicit.

> Source basis: AquaFlow Water Meter Reading & Billing System — Guided Project Roadmap, Version 1.0, September 2026. The roadmap describes 13 milestones over approximately 29 weeks and a target architecture of independent Node.js services, Next.js, PostgreSQL, RabbitMQ, Docker/Kubernetes, CI/CD and observability.